

Cloud Computing and Big Data: Fall 2026

Homework Assignment 1

Introduction:

Customer Service is a core service for a lot of businesses around the world and it is getting disrupted now by Natural Language Processing-powered applications. In this assignment, you will implement a serverless, microservice driven web application. Specifically, you will build a Dining Concierge chatbot that sends you restaurant suggestions given a set of preferences that you provide the chatbot with through conversation.

Assignment Requirements:

1. Build and deploy the frontend of the application.(10)

- A. Repurpose the following frontend starter application to interface with your chatbot. <https://github.com/aditya491929/cloud-hw1-starter>
- B. Host your frontend in an AWS S3 bucket.
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/WebsiteHosting.html>

2. Build the API for the application. (15)

- A. Use API Gateway to set up your API.
 - Use the following API/Swagger specification for your API.
<https://github.com/aditya491929/cloud-hw1-starter/blob/master/swagger/swagger.yaml>
 - Use <http://editor.swagger.io/> to visualize this file.
 - You can import the Swagger file into API Gateway
<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-documenting-api-quick-start-import-export.html>
- B. Create a Lambda function (LF0) that performs the chat operation.
 - Use the request/response model (interfaces) specified in the API specification above.
- C. For now, just implement a boilerplate response to all messages:
 - For example, if the user says anything, Bot responds: "I'm still under development. Please come back later."

Notes

1. You will need to enable CORS on your API methods
<https://docs.aws.amazon.com/apigateway/latest/developerguide/how-to-cors.html>
2. API Gateway can generate an SDK for your API, which you can use in your frontend. It will take care of calling your API, as well as session signing the API calls - an important security feature:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/generate-javascript-sdk-of-an-api.html>

3. Build a Dining Concierge chatbot using Amazon Lex. (20)

A. Create a new bot using the Amazon Lex service. To get more information about Lex read the following documentation.

<https://docs.aws.amazon.com/lex/latest/dg/getting-started.html>

B. Create a Lambda function (LF1) and use it as a code hook for Lex, which essentially entails the invocation of your Lambda before Lex responds to any of your requests – this gives you the chance to manipulate and validate parameters as well as format the bot's responses. More documentation on Lambda code hooks at the following link: <https://docs.aws.amazon.com/lex/latest/dg/using-lambda.html>

C. Bot Requirements:

i) Implement at least the following three intents:

- GreetingIntent
- ThankYouIntent
- DiningSuggestionsIntent

ii) The implementation of an intent entails its setup in Amazon Lex as well as handling its response in the Lambda function code hook. Example: for the GreetingIntent you need to-

- create the intent in Lex,
- train and test the intent in the Lex console,
- implement the handler for the GreetingIntent in the Lambda code hook, such that when you receive a request for the GreetingIntent you compose a response such as “Hi there, how can I help?”

D. For the DiningSuggestionsIntent, you need to collect at least the following pieces of information from the user, through conversation:

1. Location
2. Cuisine
3. Dining Time
4. Number of people
5. Email

E. Based on the parameters collected from the user, push the information (location, cuisine, etc.) to an SQS queue (Q1). More on SQS queues here: <https://aws.amazon.com/sqs/>

F. Also confirm to the user that you received their request and that you will notify them over email once you have the list of restaurant suggestions.

4. Integrate the Lex chatbot into your chat API (10)

- A. Use the AWS SDK to call your Lex chatbot from the API Lambda (LF0).
- B. When the API receives a request, you should:
 - i) extract the text message from the API request.
 - ii) send it to your Lex chatbot,
 - iii) wait for the response,
 - iv) send back the response from Lex as the API response.

5. Use the Yelp API to collect 1,000+ random restaurants from Manhattan (15)

Use the following tools:

A. Yelp API

- Get restaurants by your self-defined cuisine types. (min 5 cuisines)
- You can do this by adding cuisine type in the search term (ex. Term: Chinese restaurants)
- Each cuisine type should have 200 restaurants or so.
- Make sure your restaurants don't duplicate.

B. DynamoDB (a NoSQL database)

- Create a DynamoDB table and named "yelp-restaurants".
- Store the restaurants you scrape, in DynamoDB (one thing you will notice is that some restaurants might have more or less fields than others, which makes
- DynamoDB ideal for storing this data).
- With each item you store, make sure to attach a key to the object named "insertedAtTimestamp" with the value of the time and date of when you inserted
- Store fields necessary for your recommendation.
(Requirements: Business ID, Name, Address, Coordinates, Number of Reviews, Rating, Zip Code)

Note: you can perform this scraping from your computer or from your AWS account.

6. Create an OpenSearch instance using the AWS ElasticSearch Service (15)

A. Create an ElasticSearch index called "restaurants".

B. Create an ElasticSearch type under the index "restaurants" called "Restaurant".

C. Store partial information for each restaurant scraped in ElasticSearch under the "restaurants" index, where each entry has a "Restaurant" data type. You only need to store RestaurantID and Cuisine for each restaurant.

7. Build a suggestions module, that is decoupled from the Lex chatbot (15)

A. Create a new Lambda function (LF2) that acts as a queue worker.

B. Whenever it is invoked it:

- i) pulls a message from the SQS queue (Q1),
- ii) gets a random restaurant recommendation for the cuisine collected through conversation from ElasticSearch and DynamoDB,
- iii) formats them and
- iv) sends them over an email to the configured email in the SQS message, using SES (Simple Email Service).

C. Use the DynamoDB table “yelp-restaurants” (which you created earlier) to fetch more information about the restaurants (restaurant name, address, etc.), since the restaurants stored in ElasticSearch will have only a small subset of fields from each restaurant.

D. Modify the rest of the LF2 function if necessary to send the user email.

E. Set up a CloudWatch event trigger or EventBridge Scheduler that runs every minute and invokes the Lambda function (LF2). This automates the queue worker Lambda to poll and process suggestion requests on its own.

In summary, based on a conversation with the customer, your LEX chatbot will identify the customer’s preferred ‘cuisine’. You will search through OpenSearch to get random suggestions of restaurant IDs with this cuisine. At this point, you would also need to query the DynamoDB table with these restaurant IDs to find more information about the restaurants you want to suggest to your customers like name and address of the restaurant. Please note that you do not need to worry about filtering restaurants based on neighborhood in this assignment. Filter only using the cuisine.

Extra Credit (10)

- Implement a state for your concierge application, such that it remembers your last search for both location and category.
- When a user returns to the chat and requests for the same location and category as their previous search, they should be asked if they want the same recommendation as the last time.
- You can use DynamoDB to store intermediary state information to handle the recommendation based on the last search.

Example Interaction

User: Hello

Bot: Hi there, how can I help?

User: I need some restaurant suggestions.

Bot: Great. I can help you with that. What city or city area are you looking to dine in?

User: New Delhi

Bot: Sorry, I can't fulfill requests for New Delhi. Please enter a valid location.

User: Can you try Manhattan

Bot: Got it, Manhattan. What cuisine would you like to try?

User: Japanese

Bot: Ok, how many people are in your party?

User: Two

Bot: A few more to go. What date?

User: Today

Bot: What time?

User: 7 pm, please

Bot: Great. Lastly, I need your phone number so I can send you my findings.

User: 123-456-7890

Bot: You're all set. Expect my suggestions shortly! Have a good day.

User: Thank you!

Bot: You're welcome. (a few minutes later)

User gets the following Email:

"Hello! Here are my Japanese restaurant suggestions for 2 people, for today at 7pm:

1. Sushi Nakazawa, located at 23 Commerce St
2. Jin Ramen, located at 3183 Broadway
3. Nikko, located at 1280 Amsterdam Ave.

Enjoy your meal!"

Note: To reduce your OpenSearch billing cost consider using these properties while creating your domain:

Standard create: Keep the resources minimal

Environment: Dev/Test (will explain in demo)

Deployment Options: Use without standby, 1 AZ

No of Data Nodes: General purpose, t3.small.search or t3.medium.search, 1 Data node

Fine grained access control : create a master user

Further explanation of these properties will be given during class.

Note: It is very important you don't leave this domain on for a long time. It is not serverless, so you will be charged a lot of money. Ideally opensearch implementation should be done at last.

ANNEX Architecture Diagram

